TECHNICAL REPORT

YONGA ATTENTION ENGINE

A Hardware Self-Attention Accelerator on Xilinx Artix-7

Belinay Güler. Can Ertürk. M. Taha Aydemir. Hasan Sancak.
İTÜ Yonga
August 2026

github.com/ITUYonga/yonga_attention_engine

Contents

1. Introduction

Attention is one of the oldest tricks nervous systems ever learned. Long before anything had a cortex, or even a brain, organisms had already solved a version of the same problem: incoming information is cheap, but processing it is not, so something has to decide, moment to moment, what is worth paying for. A frog's retina does not forward an equally weighted account of everything in its visual field to the rest of its nervous system. It forwards the fly. That same asymmetry, weighting some information heavily and the rest barely at all, reappears billions of years later at the center of every modern large language model.

Self-attention is the mechanism transformer models use to decide, for every token in a sequence, which other tokens actually deserve its attention, and by how much. Each token is projected into a Query, a Key, and a Value vector, similarity is measured between every pair of tokens by a dot product of Query and Key, and those similarity scores are turned into a weighted sum over Value vectors by softmax: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d\_k} + \text{mask}) \cdot V$. It is a small amount of arithmetic, repeated at scale, and it is most of what makes a modern language model work at all.

That formula usually runs as a handful of matrix multiplications on a GPU, several layers of software away from the transistors doing the work. Yonga Attention Engine asks a narrower question: what does that same operation look like written directly as digital logic, with no soft-core processor executing instructions and no vendor-supplied neural network block doing the actual computation? Every multiply, every accumulation, and every exponential in the pipeline described in this report is RTL the team wrote, simulated, and can account for.

This report documents the resulting design in three parts. Sections 2 and 3 describe what was built and why it is organized the way it is. Sections 4 and 5 describe how it was checked and what it costs on real hardware. Sections 6 and 7 describe what happened when four independently written pieces had to work as a single chip, and what is still open.

2. Overview

Yonga Attention Engine is a from-scratch register-transfer-level implementation of transformer self-attention: rotary position embeddings, grouped query attention, and scaled dot-product attention with softmax, written directly in synthesizable SystemVerilog and running on a single mid-range FPGA. The goal was to take the mathematical definition of attention and implement it in hardware without a soft-core processor or a vendor IP block doing the actual computation: every multiply, every accumulation, and every exponential in the pipeline is logic the team wrote and can account for.

Arithmetic throughout the design is bfloat16 (BF16), chosen for its wide dynamic range and its cheap conversion to and from the FP32 values used at the AXI-Stream boundary. Four people designed, verified, and merged their own modules into a single RTL tree. This report documents the resulting architecture, the testbench-based verification used to check it, and the resource cost of running it on real hardware.

3. System Architecture

The design is organized as a streaming pipeline. Tokens arrive one at a time over AXI-Stream, are projected into query, key, and value vectors, rotated by RoPE, mapped from query heads to key/value heads by the GQA mapper, and fed into a shared systolic array that computes both the $Q \cdot K^T$ similarity

scores and, after softmax, the score·V weighted sum. The result is projected back out and converted to FP32 on its way off-chip.

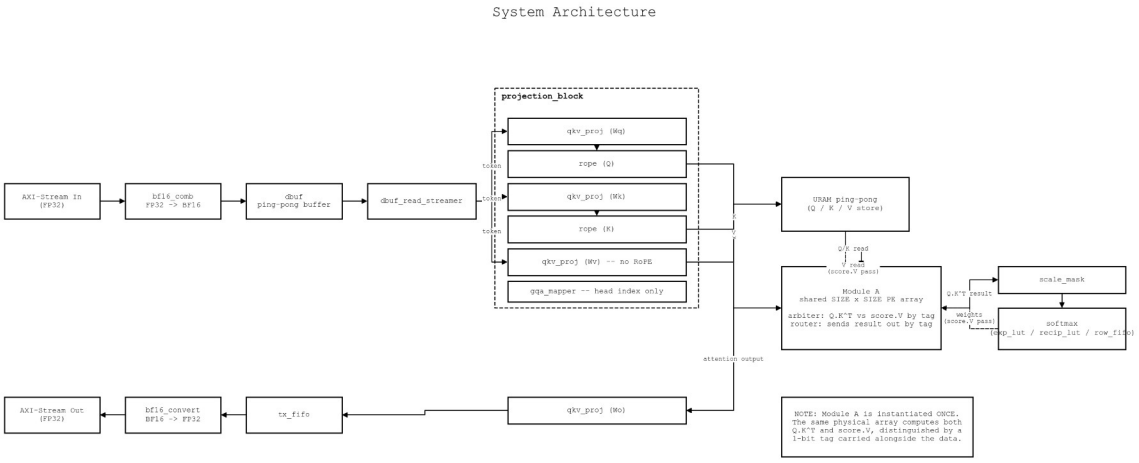

Figure 1. Top-level data flow of Yonga Attention Engine.

The array at the center of the pipeline, referred to internally as Module A, is instantiated exactly once. The same physical grid of processing elements computes $Q \cdot K^T$ on one pass and score·V on the next, distinguishing the two data streams with a one-bit tag carried alongside every value. Reusing one array for both matrix products, rather than building two, is the single largest resource-saving decision in the design.

QKV Projection

qkv_proj.sv is a single parameterized matrix-vector multiply module, instantiated four times (Wq, Wk, Wv, Wo) with a different weight matrix loaded into each. It is implemented as a state machine reusing one multiplier and one adder per instance rather than a parallel array: correctness first, with a wider datapath left as a follow-up once the team confirms it is actually the bottleneck.

Rotary Position Embedding (RoPE)

rope.sv rotates the query and key vectors, never the value vector, using sine and cosine values read from a ROM addressed by position and pair index. Because the datapath has no dedicated subtractor, the “minus” in the rotation formula is implemented by flipping the sign bit of the second operand before it reaches the shared adder, avoiding a second arithmetic unit for a case that already reduces to addition.

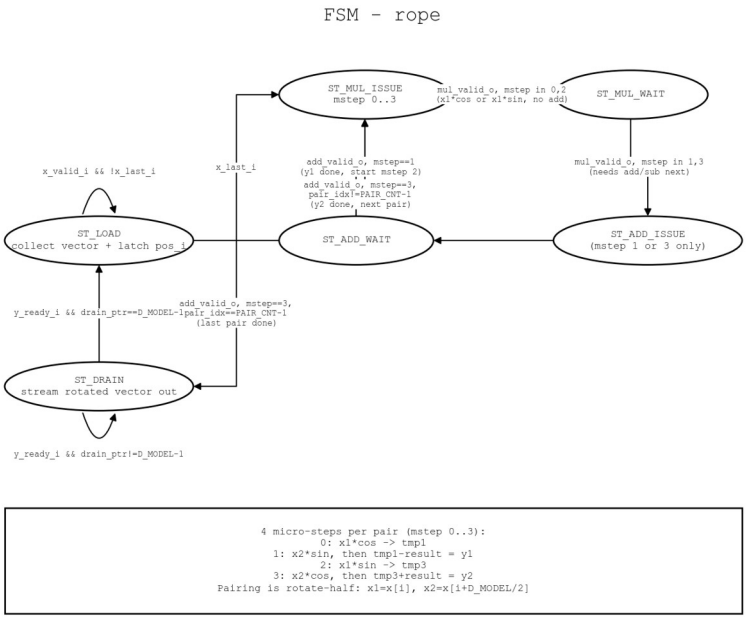

Figure 2. RoPE control flow: ROM lookup, rotate-half, and the shared adder path.

Grouped Query Attention (GQA) Mapping

gqa_mapper.sv is a purely combinational index translation: given the current query head, it returns which key/value head that query head should read from. It never touches vector data itself, which keeps the head-mapping logic independent of, and reusable across, whatever arithmetic width the rest of the pipeline settles on.

Shared Systolic Array (Module A)

The array is a SIZE×SIZE output-stationary systolic grid. Query and key elements stream in serially along the depth dimension, and once a full depth slice has accumulated, the array switches to a parallel read-out phase. An input arbiter gives priority to score·V requests arriving back from softmax over new $Q \cdot K^T$ tiles, which keeps the softmax-to-output path from stalling behind newly arriving queries, and an output router sends each result back to softmax or on to the output projection depending on the same tag bit.

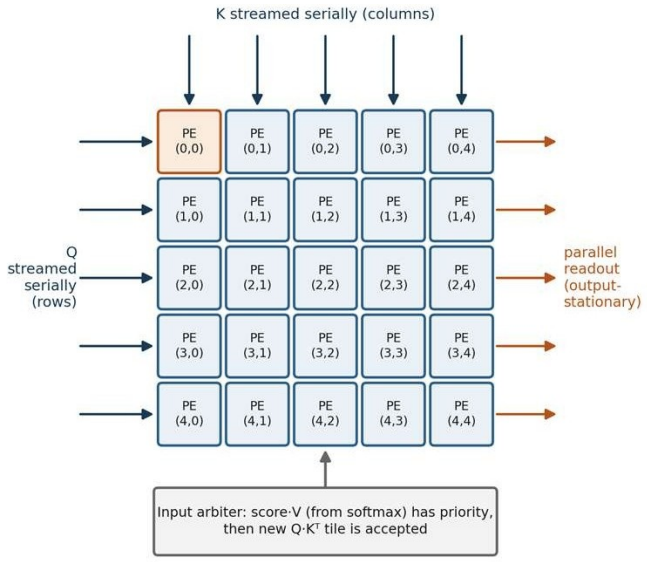

Figure 3. Output-stationary operation of the shared Q·K^T / score·V systolic array.

Scale, Mask and Softmax

Raw similarity scores are scaled and, per element, masked to negative infinity in a three-stage backpressure pipeline (scale_mask.sv), then normalized by a four-state softmax finite state machine (softmax.sv): ACCUMULATE streams each row element through a LUT-based exponential and into a running sum, INVERT turns that sum into a reciprocal through a second LUT, DIVIDE_NORMALIZE streams the row back through multiplying by the reciprocal, and IDLE waits for the next row. Both the exponential and the reciprocal come from small combinational LUTs rather than an iterative approximation, trading a small amount of numerical accuracy for a pipeline that never stalls waiting on convergence.

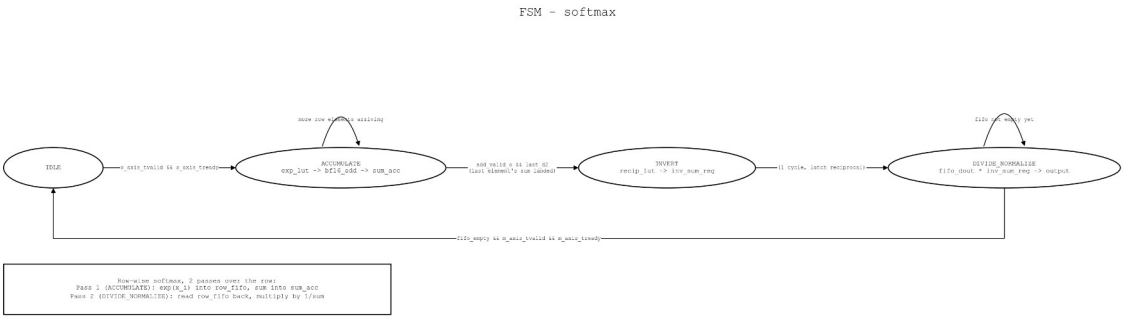

Figure 4. The four-state softmax finite state machine.

Memory and Data Routing

dbuf.sv lets a new token vector begin loading while the previous one is still being processed. Q, K, and V are held in ping-pong URAM banks, so Module A can read one bank while the next token's projections write the other. tx_fifo.sv and axi_stream_if.sv carry finished output vectors back out over AXI-Stream, performing the BF16-to-FP32 conversion at the boundary.

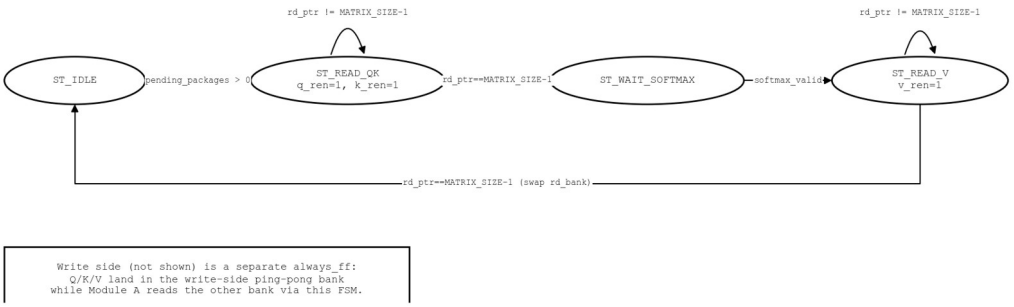

Figure 5. Ping-pong URAM bank switching between Module A reads and QKV projection writes.

4. Verification

Every module has a directed SystemVerilog testbench under tb/, summarized in Table 1. All seven were run to completion in Vivado XSim against golden reference values computed independently in Python. Several of the bugs the testbenches caught were timing bugs invisible to static reading of the RTL, including a systolic-array stimulus loop that assumed a single ready pulse guaranteed acceptance, and a softmax pipeline that silently dropped one row element in four because a valid flag failed to clear when its enable condition went false.

Table 1. Testbench coverage and results.

Testbench	Covers	Result
tb_bf16_add.sv	Pipelined BF16 adder	PASS 7/7
tb_bf16_mul.sv	Pipelined BF16 multiplier	PASS 6/6
tb_gqa_mapper.sv	Query-head to KV-head mapping	PASS 8/8
tb_qkv_proj.sv	Weight loading and W·x MAC	PASS 2/2
tb_rope.sv	ROM load, rotate-half, sin/cos check	PASS
tb_qk_array.sv	Shared systolic array (mod_a_wrapper)	PASS 4/4
tb_scale_mask_softmax.sv	Scale, mask, softmax over two rows	PASS 8/8

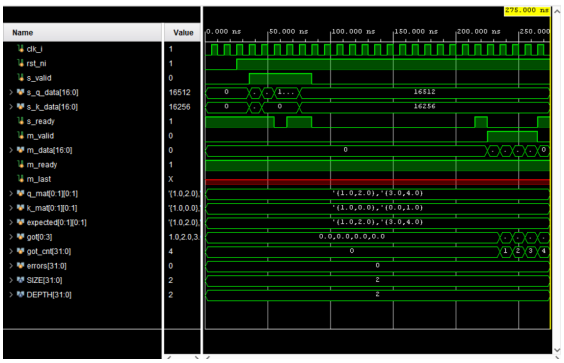

Figure 6a. tb_qk_array.sv

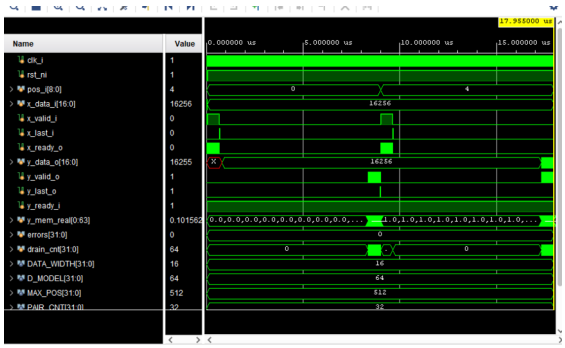

Figure 6b. tb_rope.sv

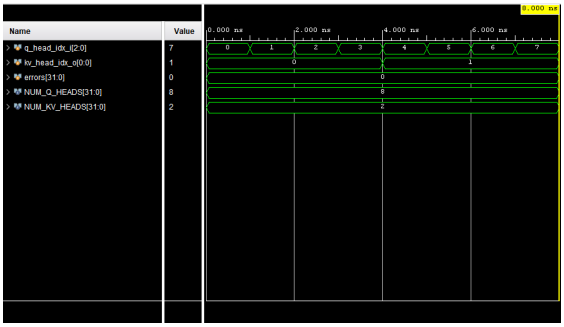

Figure 6c. tb_gqa_mapper.sv

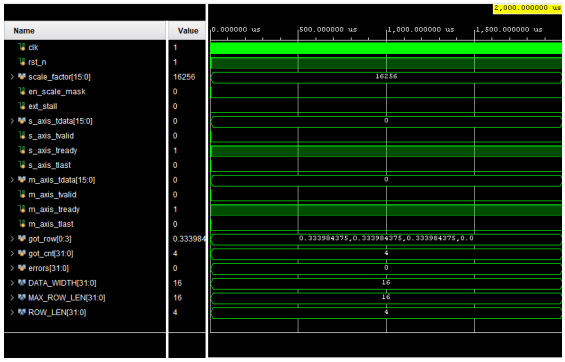

Figure 6d. tb_scale_mask_softmax.sv

Figure 6. Representative waveforms from four of the seven testbenches.

5. Resource Utilization

Post-implementation synthesis targets a Xilinx Artix-7 device (Nexys4 DDR, xc7a100tcsg324-1) in Vivado 2024.1. The design uses no DSP48E1 slices and no Block RAM at all, consistent with the goal of a portable, LUT-only BF16 arithmetic core: the majority of LUT usage is distributed RAM for weight and ROM storage rather than logic.

Table 2. Post-implementation resource utilization.

Resource	Used	Available	Utilization
Slice LUTs	1,486	63,400	2.34%
Slice Registers	510	126,800	0.40%
Block RAM	0	135	0.00%
DSP48E1	0	240	0.00%

A single port-width bug during integration, an AXI-Stream data output declared 16 bits wide instead of 32, was enough for Vivado's synthesizer to prove the entire output path carried no usable information and quietly optimize the design from roughly 1,500 real LUTs down to 73. Correcting that one port restored the full pipeline to the numbers above, and is the reason the team now treats a clean post-synthesis utilization report, not just a passing simulation, as part of verifying a merge.

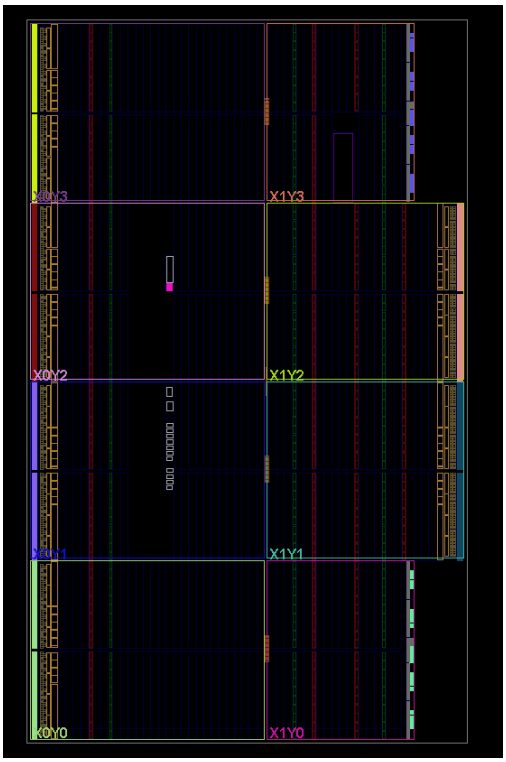

Figure 7. Post-implementation device floorplan on the Nexys4 DDR (xc7a100tcsg324-1).

6. Integration

None of the four members' RTL lived in the same place before the integration stage: each person's modules were written and simulated in isolation, against assumptions about the others' interfaces that had never been checked against real code. Merging everything into the single rtl/ tree in this repository surfaced a list of concrete interface mismatches that reading the code side by side would not have caught. Elaborating the whole design in Vivado did: a ping-pong controller expecting a memory port that the memory module never exposed, an internally-computed “last element” signal that never reached the top-level port list, and the port-width bug described in Section 5. Each was found by trying to build the whole chip, not by reviewing any one module on its own.

7. Current Limitations

- RoPE's rotation math is exercised by tb_rope.sv but not yet cross-checked against a reference implementation over a wide sweep of positions.
- QKV projection reuses a single multiplier and adder per weight matrix rather than a parallel datapath, which caps throughput until the team confirms a wider design is actually needed.
- The LUT-based exponential and reciprocal in softmax trade numerical accuracy for pipeline simplicity, and the size of that error has not yet been characterized against a floating-point softmax over realistic score distributions.
- End-to-end, multi-token throughput on hardware has not yet been measured: current verification is per-module, in simulation.

Repository: github.com/ITUYonga/yonga_attention_engine